Journal of Applied Data Sciences
Vol. 4, No. 4, Juni 2025, pp.
414-430

Sistem Deteksi Emosi pada Teks Menggunakan Metode LSTM

Akbyna Riyomi [1], Cindy Ardelya [2, *], Fatharony Fatwa [3,]

[1,2,4] *Information Technology, Politeknik Caltex Riau, Jalan Umban Sari No.1, Pekanbaru, Riau 28265, Indonesia*

(Received: June 13, 2025; Revised: July 5, 2025; Accepted: July 27, 2025; Available online: August 10, 2025)

Abstract

Perkembangan media sosial telah menjadikan teks sebagai salah satu bentuk utama ekspresi pengguna, termasuk dalam mengungkapkan emosi dan kondisi psikologis seperti stres. Penelitian ini bertujuan untuk mengembangkan sistem deteksi emosi berbasis analisis teks menggunakan model *Long Short-Term Memory (LSTM)*. Dataset yang digunakan dalam penelitian ini diperoleh dari platform Kaggle dan mencakup data teks dengan berbagai label emosi dalam dua bahasa. Proses pengolahan data mencakup tahapan pembersihan, tokenisasi, normalisasi, dan pelabelan emosi. Sistem yang dibangun diharapkan mampu mengidentifikasi emosi dalam teks secara akurat, sehingga dapat diterapkan dalam chatbot konseling atau sistem monitoring psikologis berbasis teks.

Keywords: Deteksi Emosi, Analisis Teks, LSTM.

1. Introduction

Perkembangan teknologi informasi dan komunikasi, khususnya media sosial, telah mengubah cara manusia berinteraksi dan mengekspresikan emosi mereka. Platform seperti Twitter, Facebook, dan Instagram memungkinkan pengguna untuk membagikan pemikiran, pengalaman, serta keluh kesah secara real-time melalui teks. Unggahan-unggahan tersebut tidak hanya berfungsi sebagai sarana komunikasi, tetapi juga dapat merefleksikan kondisi emosional pengguna, seperti perasaan sedih, marah, cemas, bahagia, dan berbagai emosi lainnya.

Fenomena ini membuka peluang penelitian yang besar, khususnya dalam bidang pemrosesan bahasa alami (Natural Language Processing/NLP), untuk membangun sistem yang mampu mendeteksi emosi secara otomatis dari teks yang diunggah pengguna. Sistem semacam ini memiliki potensi untuk diterapkan dalam berbagai bidang, termasuk analisis media sosial, peningkatan layanan pelanggan, serta pengembangan sistem pendukung kesehatan mental. Namun, deteksi emosi berbasis teks bukanlah tugas yang sederhana. Bahasa manusia sangat kompleks, di mana emosi dapat diungkapkan secara eksplisit maupun implisit, dengan penggunaan kata yang memiliki makna ganda, variasi gaya bahasa, serta perbedaan konteks budaya. Oleh karena itu, diperlukan metode yang mampu memahami pola urutan kata dan menangkap konteks secara mendalam.

Long Short-Term Memory (LSTM) merupakan salah satu metode deep learning yang terbukti efektif dalam mengolah data sekuensial seperti teks. LSTM memiliki kemampuan untuk mengingat informasi penting dalam urutan kata, sehingga dapat memahami makna kalimat dengan lebih akurat dibandingkan metode konvensional. Dengan keunggulan tersebut, penelitian ini memilih LSTM sebagai metode utama dalam membangun sistem deteksi emosi pada teks.

Penelitian ini bertujuan untuk mengembangkan sistem deteksi emosi berbasis LSTM dengan memanfaatkan dataset publik yang dikumpulkan dari platform Kaggle. Dataset yang digunakan terdiri dari teks berbahasa Indonesia dan Inggris yang telah diberi label emosi seperti sedih, marah, cemas, bahagia, dan lainnya. Sebelum digunakan pada model, data akan melalui serangkaian proses pra-pemrosesan seperti pembersihan teks, tokenisasi, normalisasi, dan penghapusan kata tidak penting (stopwords).
Dengan sistem yang dikembangkan, diharapkan penelitian ini dapat memberikan kontribusi pada pengembangan teknologi kecerdasan buatan, khususnya dalam analisis emosi berbasis teks, yang bermanfaat untuk penelitian akademis, layanan publik, serta sistem deteksi dini kondisi emosional pengguna.

2. Research Methodology

Berdasarkan uraian sebelumnya, penelitian ini mengeksplorasi domain deteksi emosi pada teks media sosial menggunakan pendekatan Long Short-Term Memory (LSTM), yang terbukti efektif untuk memproses data sekuensial. Penelitian ini memanfaatkan dataset publik yang berisi kumpulan teks berbahasa Indonesia dan Inggris dengan label emosi (happy, anger, sadness, love, dan fear) yang diperoleh dari platform Kaggle.

Metodologi penelitian ini menjelaskan langkah-langkah yang dilakukan untuk membangun sistem deteksi emosi pada teks media sosial menggunakan metode Long Short-Term Memory (LSTM) :

1) Analisis Kebutuhan Penelitian

Tahap ini meliputi penentuan tujuan penelitian, pemilihan dataset, dan identifikasi emosi yang akan diklasifikasikan (happy, anger, sadness, love, fear). Dataset yang digunakan diperoleh dari Kaggle dengan format CSV yang telah memiliki label emosi.

2) Pra-Pemrosesan Data (Text Pre-processing)

Data mentah dari dataset akan dibersihkan dengan langkah-langkah berikut:

- Cleaning: Menghapus URL, mention (@username), hashtag, angka, simbol, dan karakter non-alfabet.
- Case Folding: Mengubah semua huruf menjadi huruf kecil.
- Stopword Removal: Menghapus kata-kata umum yang tidak memiliki makna signifikan.
- Stemming dan Normalisasi: Mengubah kata menjadi bentuk dasar dan mengganti kata gaul (slang) dengan kata baku menggunakan kamus khusus.

Hasil dari tahap ini berupa teks bersih dan siap diubah menjadi representasi numerik.

3) Tokenisasi dan Padding

Setelah pembersihan, teks diubah menjadi deretan angka (token) menggunakan Tokenizer Keras. Untuk memastikan panjang input seragam, diterapkan padding pada setiap urutan kata dengan panjang maksimum 100 kata.

4) Pembangunan Model LSTM

Model dibangun menggunakan TensorFlow dan Keras dengan arsitektur:

- Embedding Layer: input_dim=5000, output_dim=128, input_length=100.
-
- LSTM Layer: 64 unit.
- Dropout Layer: 0.5 untuk mencegah overfitting.
- Dense Layer: 32 unit (aktivasi ReLU).
- Output Layer: Softmax (untuk 5 kelas emosi).

5) Pembagian Data (Data Split)

Dataset dibagi menjadi 80% data latih dan 20% data uji menggunakan metode stratified split untuk menjaga proporsi label.

6) Pelatihan Model (Training)

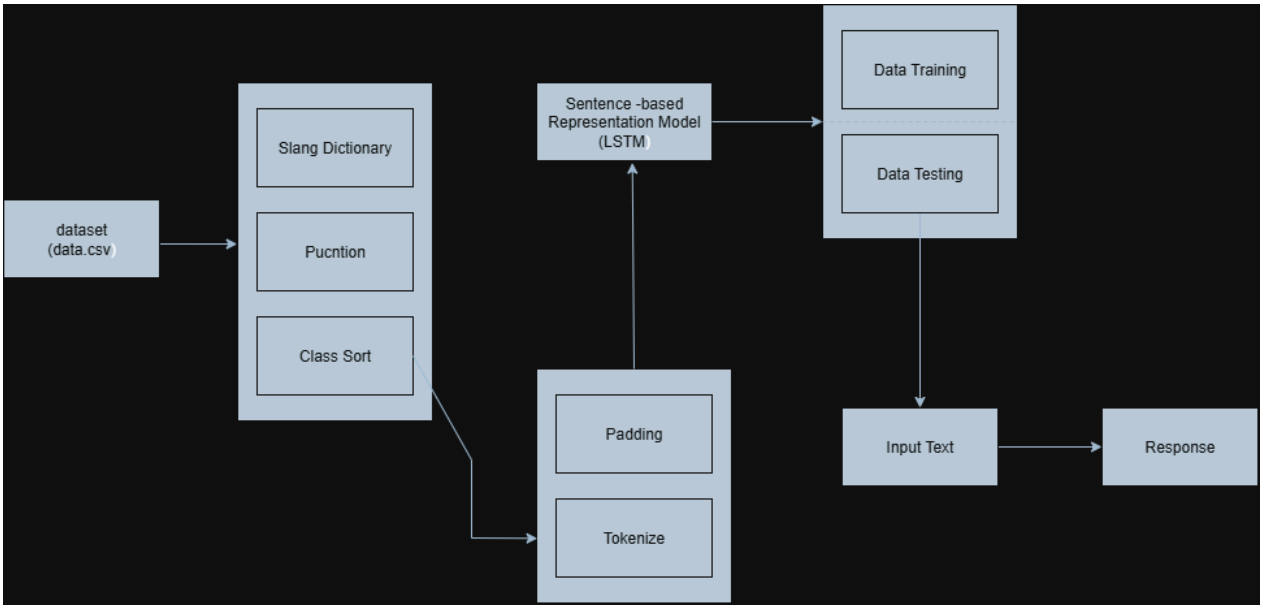
Model LSTM dilatih selama 6 epoch dengan batch size 32 dan optimisasi Adam. Validasi dilakukan pada data uji (X_test, y_test).

7) Evaluasi Model

Evaluasi dilakukan menggunakan metrik akurasi, precision, recall, F1-score, serta confusion matrix. Hasil evaluasi digunakan untuk mengukur performa model dalam mengklasifikasikan emosi.

2.1. Arsitektur Sistem

Arsitektur sistem deteksi emosi yang diusulkan ditunjukkan pada Gambar 2. Alur sistem dapat dijelaskan sebagai berikut:



2.1 Arsitektur Sistem

Adapun penjelasan dari arsitektur yang digunakan pada Gambar di atas adalah sebagai berikut:

1) Proses dimulai dari dataset (data.csv), yaitu sebuah kumpulan data teks yang digunakan sebagai sumber informasi. Dataset ini nantinya akan diproses lebih lanjut sebelum dapat digunakan pada tahapan selanjutnya.
2) Sebelum melangkah ke proses pembelajaran, teks mentah pada dataset akan melalui tahapan preprocessing. Dalam tahapan ini, pertama-tama terjadi proses penghapusan tanda baca (punctuation) untuk membersihkan kalimat dari karakter-karakter yang tidak relevan. Setelah itu, slang dictionary diterapkan untuk menormalkan bahasa gaul atau kata tidak baku menjadi kata baku. Langkah selanjutnya adalah lemmatization, yaitu proses menemukan bentuk kata dasarnya, sehingga dapat menyederhanakan variasi kata yang terjadi. Setelah proses lemmatization, kalimat kemudian dikelompok berdasarkan kelas (Class Sort) sesuai label yang tersedia.
3) Selanjutnya, pada tahapan Tokenize, kalimat yang masih berupa teks diubah menjadi sebuah deret nomor atau indeks. Hal ini berguna agar dapat diproses lebih lanjut oleh model. Setelah proses Tokenize, diterapkan Padding, yaitu menyamakan ukuran kalimat dengan cara menambah padding pada kalimat yang lebih pendek. Dengan begitu, ukuran kalimat menjadi seragam dan siap dimasukkan ke model.
4) Kalimat yang telah diberi padding kemudian diteruskan ke Sentence-based Representation Model (LSTM). Dalam proses ini, LSTM belajar dari urutan kata dan konteks kalimat untuk mendeteksi emosi .
5) Sebelum melatih model, dataset dibagi menjadi dua, yaitu data training dan data testing, demi menjaga kualitas dan validitas model. Setelah proses pelatihan, model yang dihasilkan nantinya dapat digunakan untuk menganalisa kalimat input dan memberikan respon berupa deteksi emosi berdasarkan kalimat yang dimasukkan.

2.1. Dataset

Pengumpulan data merupakan tahap awal dalam pengembangan sistem deteksi emosi berbasis analisis teks. Dalam penelitian ini, data dikumpulkan dari platform terbuka berupa kumpulan teks media sosial, yang diunduh dari Kaggle. Dataset terdiri dari tweet dalam bahasa Indonesia dan Inggris yang telah dilabeli berdasarkan kategori emosi seperti marah, senang, takut, sedih, dan lainnya.

Selain itu, data juga mencakup berbagai ekspresi pengguna yang mencerminkan kondisi emosional atau potensi stres, sehingga dapat dimanfaatkan untuk pelatihan dan pengujian model. Format data telah disusun dalam bentuk *CSV* yang berisi pasangan antara teks (tweet) dan label emosi. Dataset ini kemudian dikonversi ke format yang sesuai untuk pemrosesan lebih lanjut dan pelatihan model, seperti dalam format JSON atau array terstruktur.

label		tweet
anger happy Other (2283)	25% 23% 52%	**4389** unique values
anger		Soal jln Jatibaru,polisi tdk bs GERTAK gubernur .Emangny polisi tdk ikut pmbhasan? Jgn berpolitik. P...

3.1 Dataset Indonesia Twitter

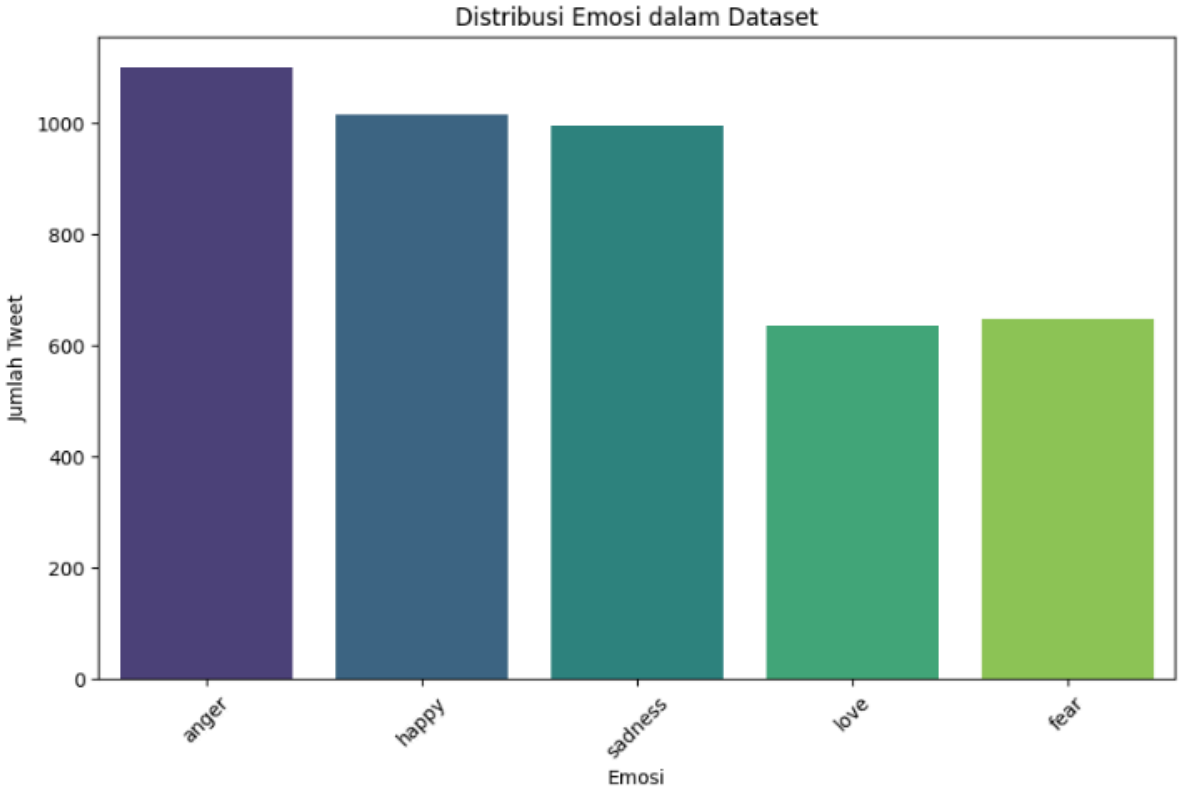
3. Results and Discussions

Bab ini membahas hasil penelitian, meliputi analisis dataset, tahap pra-pemrosesan data, pembangunan serta pelatihan model LSTM, dan evaluasi performa model. Seluruh proses ini dirancang untuk menghasilkan sistem deteksi emosi berbasis analisis teks dengan tingkat akurasi yang optimal.

3.1. Dataset

Dataset yang digunakan dalam penelitian ini diperoleh dari Kaggle, berisi kumpulan tweet berbahasa Indonesia dan Inggris dengan label emosi seperti anger, happy, sadness, love, dan fear. Dataset disimpan dalam format CSV dengan dua kolom utama: tweet (teks) dan label (kategori emosi).

Distribusi jumlah data di setiap label ditunjukkan pada Gambar 1. Dari gambar terlihat bahwa anger memiliki jumlah data terbanyak (lebih dari 1.000 tweet), diikuti oleh happy dan sadness. Sedangkan love dan fear memiliki jumlah data paling sedikit (sekitar 600-an tweet).



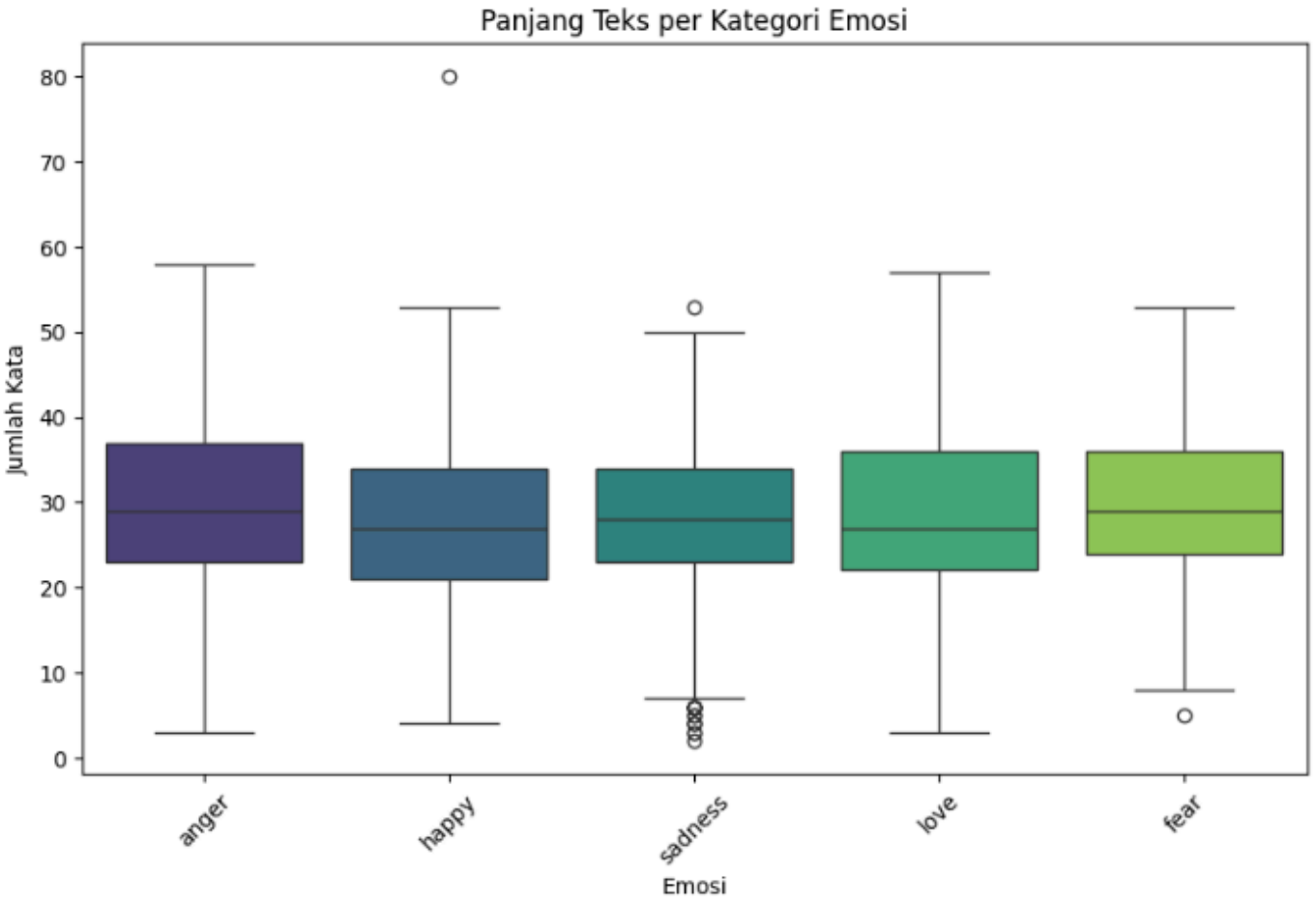
Gambar 3.1 Distribusi Emosi dalam Dataset

3.2. Analisis Panjang Text

Analisis panjang teks dilakukan untuk mengetahui sebaran jumlah kata dalam tweet pada setiap kategori emosi. Hasil visualisasi menggunakan boxplot menunjukkan bahwa:

1) Sebagian besar tweet memiliki panjang antara 20–35 kata.
2) Kategori love memiliki variasi panjang teks yang paling beragam (terlihat dari boxplot dengan rentang yang lebih lebar).
3) Beberapa outlier terdapat pada kategori happy dan sadness, dengan panjang tweet mencapai lebih dari 70 kata.
4) Visualisasi boxplot ditunjukkan pada Gambar 2.

Visualisasi boxplot ditunjukkan pada **Gambar 3.2**.



Gambar 3.2 Distribusi Emosi dalam Dataset

3.3. Text Processing

Tahapan pra-pemrosesan teks dilakukan untuk membersihkan data mentah sehingga siap diproses oleh model.

Kode Cleaning Teks :

Listing 1. Cleaning Teks

```
def clean_text(text):
    text = text.lower()
    text = re.sub(r"http\S+|@\w+|#\w+|\d+|[^\w\s]", "", text)
    text = re.sub(r"\s+", " ", text).strip()
    text = " ".join([word for word in text.split() if word not in stop_words])
    return text

df["clean_text"] = df["tweet"].astype(str).apply(clean_text)
```

3.4. WordCloud

Untuk mengetahui kata-kata dominan di setiap kategori emosi, dilakukan visualisasi WordCloud.

Hasil WordCloud menunjukkan bahwa kata "bahagia" dan "senang" paling sering muncul pada kategori happy, sedangkan kategori anger didominasi kata "kesal" dan "marah".

Kode WordCloud :

Listing 2. WordCloud untuk tiap emosi.

```
def generate_wordcloud(text, title):
    wordcloud = WordCloud(width=800, height=400,
                          background_color='white',
                          stopwords=stop_words).generate(text)
    plt.figure(figsize=(10,5))
    plt.imshow(wordcloud)
    plt.title(title, size=15)
    plt.axis('off')
    plt.show()

for emotion in df['label'].unique():
    text = ' '.join(df[df['label']==emotion]['tweet'].tolist())
    generate_wordcloud(text, f'Kata yang Sering Muncul - Emosi: {emotion}')
```

3.5. Pembangunan Model LSTM

Model dibangun dengan arsitektur sebagai berikut:

- Embedding Layer: input_dim=5000, output_dim=128, input_length=100.
- LSTM Layer: 64 unit.
- Dropout Layer: 0.5.
- Dense Layer: 32 unit (ReLU).
- Output Layer: Softmax (5 kelas emosi).

Kode Model :

Listing 3. Arsitektur LSTM.

```
model = Sequential([
    Embedding(input_dim=5000, output_dim=128, input_length=100),
    LSTM(64, return_sequences=False),
    Dropout(0.5),
    Dense(32, activation='relu'),
    Dense(len(le.classes_), activation='softmax')
])
```

```
model.compile(loss='sparse_categorical_crossentropy',
              optimizer='adam',
              metrics=['accuracy'])
```

3.6. Pelatihan Model

Model dilatih dengan 6 epoch, batch size=32, dan validasi pada data uji.

Listing 4. Proses pelatihan model.

```
model.fit(X_train, y_train,
          epochs=6,
          batch_size=32,
          validation_data=(X_test, y_test))
```

3.7. Evaluasi Model

Evaluasi menggunakan classification report dan confusion matrix.

Listing 5. Evaluasi model.

```
y_pred = model.predict(X_test)
y_pred_classes = np.argmax(y_pred, axis=1)
print(classification_report(y_test, y_pred_classes, target_names=le.classes_))
```

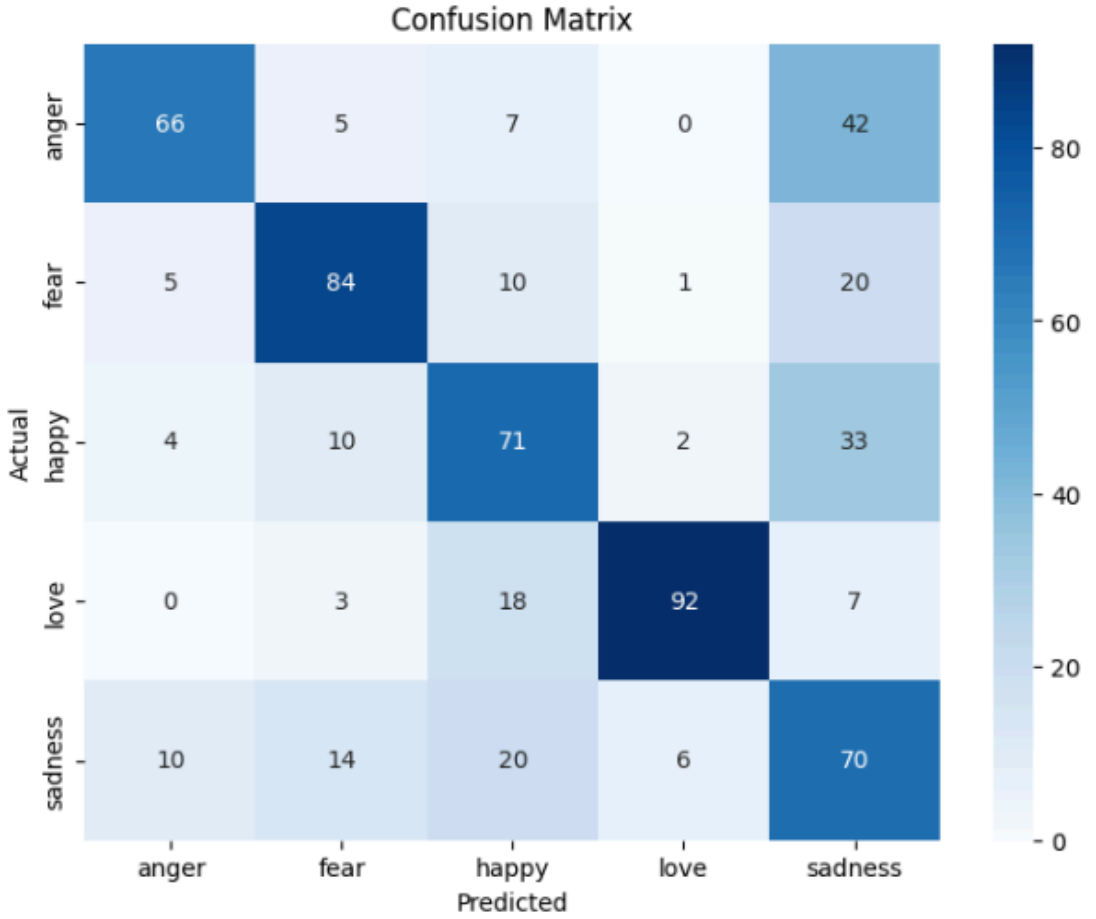
Table 1. Hasil Evaluasi Model LSTM berdasarkan Precision, Recall, dan F1-Score

Emosi	Precision	Recall	F1-score	Support
Anger	0.78	0.55	0.64	120
Fear	0.72	0.70	0.71	120
Happy	0.56	0.59	0.58	120
Love	0.91	0.77	0.83	120
Sadness	0.41	0.58	0.48	200
Accuracy	-	-	**0.64**	600
macro avg	0.68	0.64	0.65	600
weighted avg	0.68	0.64	0.65	600

Confusion Matrix:

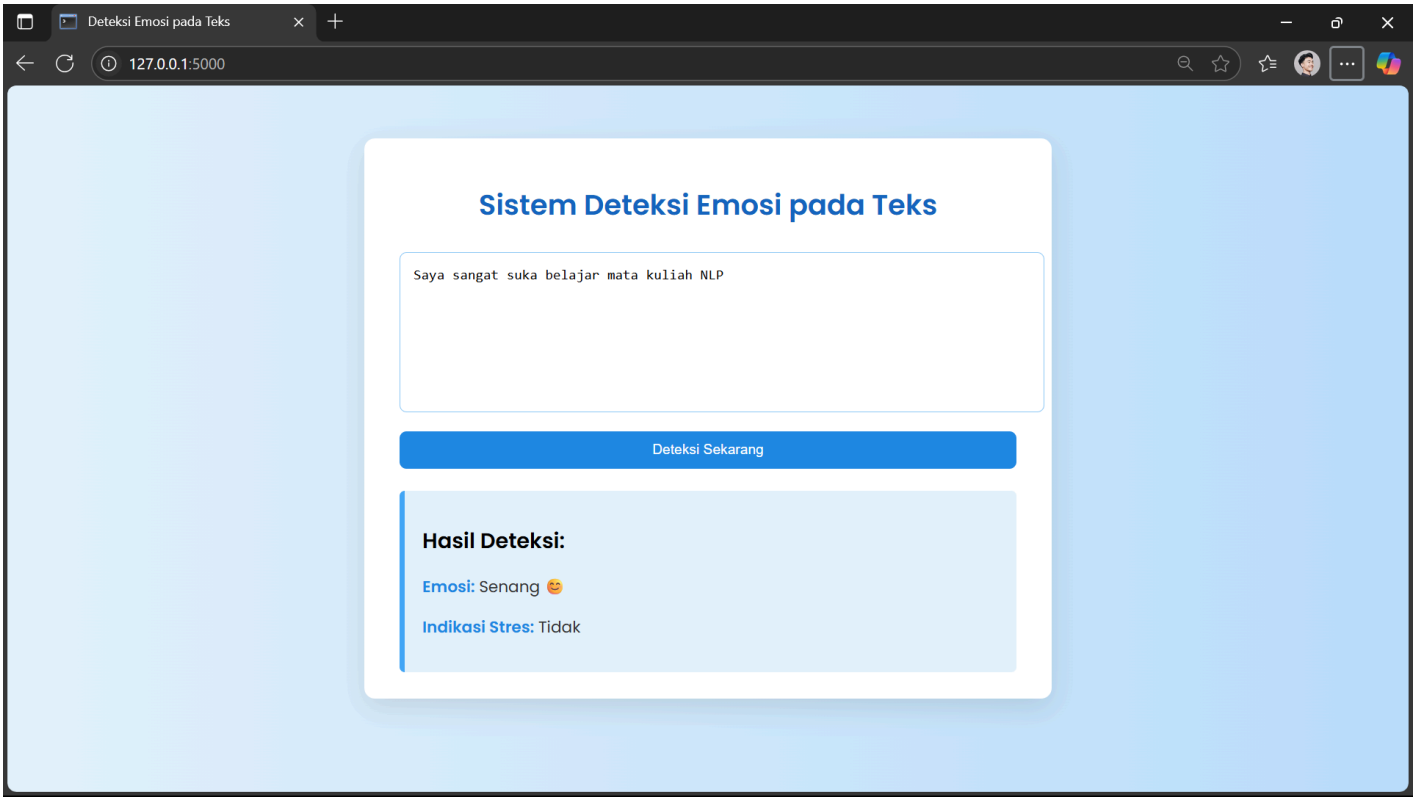
Listing 6. Visualisasi confusion matrix.

```
cm = confusion_matrix(y_test, y_pred_classes)
plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```

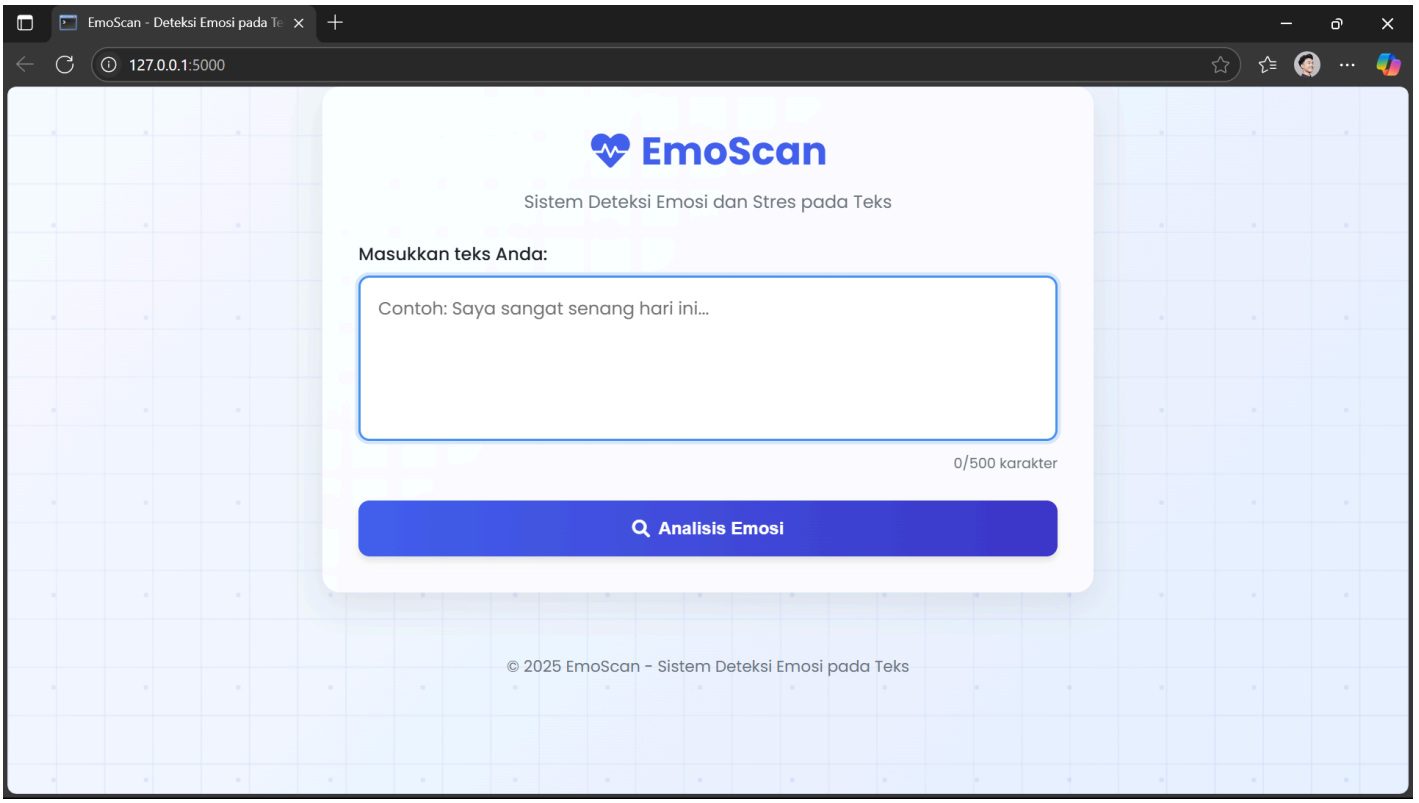


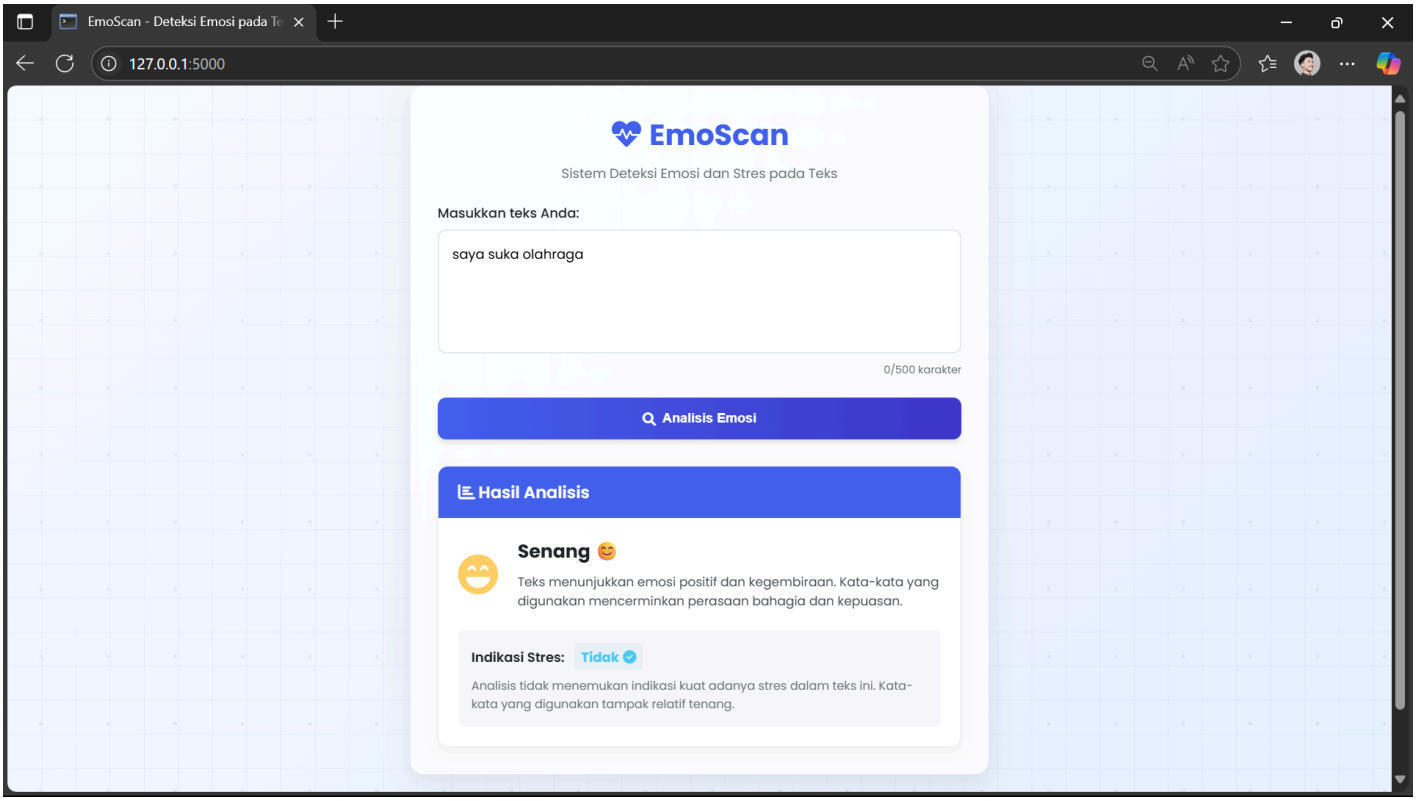
Gambar 3.3 Visualisasi Confusion Matrix

3.8. Implementasi Website

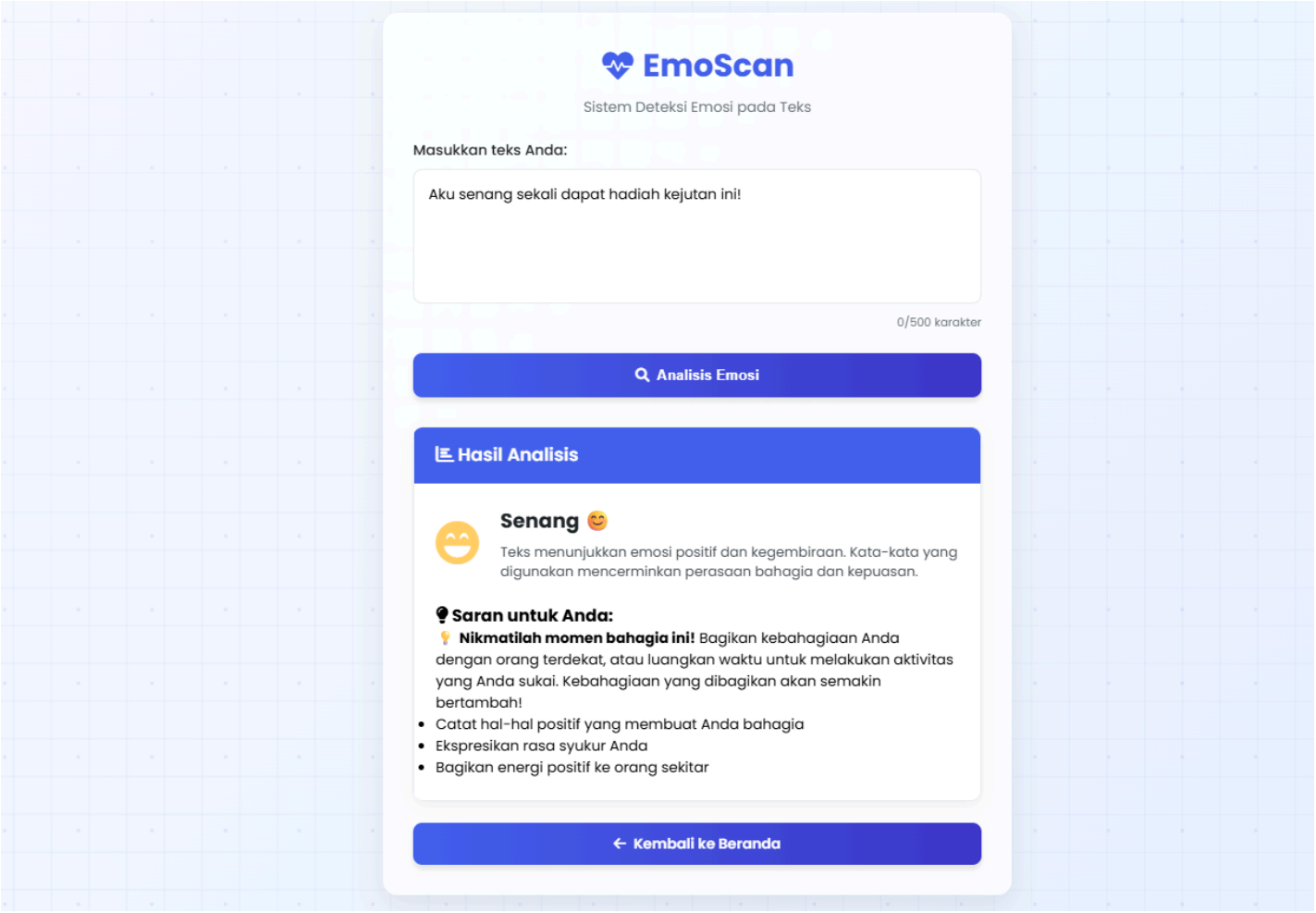


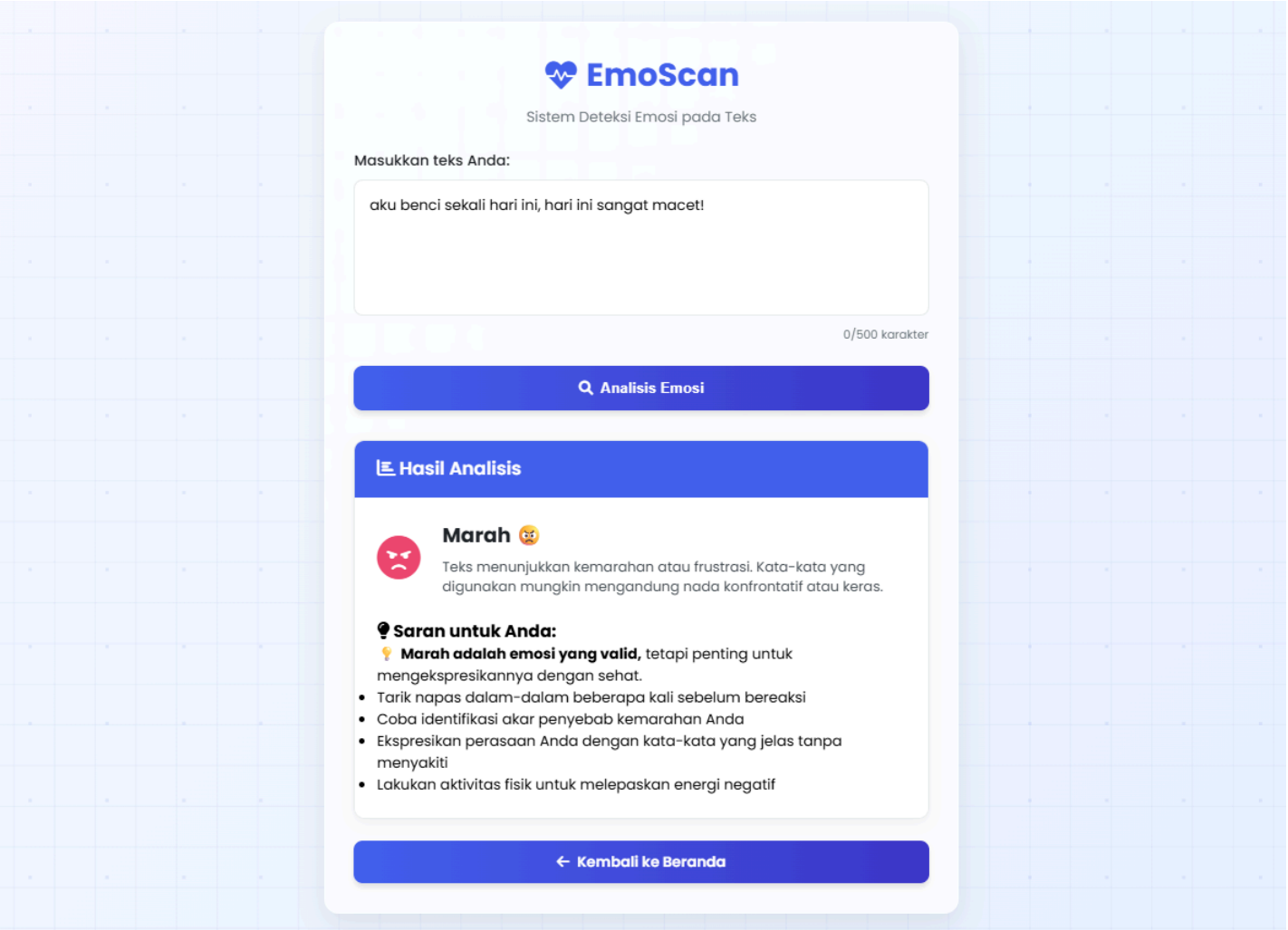
Analisa : Tampilan website pada gambar yang Anda lampirkan sudah terlihat rapi, bersih, dan modern dengan nuansa biru muda yang menenangkan, cocok untuk konteks aplikasi deteksi emosi. Elemen-elemen seperti judul, input teks, tombol aksi, dan hasil deteksi ditampilkan secara terstruktur dalam satu kolom tengah yang responsif, serta penggunaan warna biru pada tombol dan kotak hasil memberikan kesan profesional. Font yang digunakan mudah dibaca, dan penggunaan emoji pada hasil emosi menambah sentuhan humanis yang ramah pengguna. Secara keseluruhan, desain ini sudah sangat layak digunakan dan enak dipandang.





Analisa : Tampilan baru EmoScan tetap mempertahankan kesan bersih dan profesional dengan dominasi putih dan aksen biru, namun kini lebih minimalis dan fokus pada fungsionalitas. Perubahan utama terlihat pada penyederhanaan layout dengan menghilangkan gradien biru muda yang sebelumnya ada, digantikan dengan background putih polos yang membuat konten lebih menonjol. Fitur baru seperti penghitung karakter (0/500) ditambahkan untuk membantu pengguna, sementara hasil analisis emosi dan stres kini ditampilkan lebih terstruktur dengan penjelasan deskriptif yang lebih rinci. Meski beberapa elemen seperti emoji dihilangkan, tampilan baru ini terasa lebih ringkas dan langsung pada inti, dengan hierarki informasi yang lebih jelas sehingga pengguna bisa lebih fokus memahami hasil deteksi emosi mereka.


EmoScan
Sistem Deteksi Emosi pada Teks
Masukkan teks Anda:
Aku senang sekali dapat hadiah kejutan ini!
0/500 karakter
Analisis Emosi
Hasil Analisis
Senang
Teks menunjukkan emosi positif dan kegembiraan. Kata-kata yang digunakan mencerminkan perasaan bahagia dan kepuasan.
Saran untuk Anda:
Nikmatilah momen bahagia ini! Bagikan kebahagiaan Anda dengan orang terdekat, atau luangkan waktu untuk melakukan aktivitas yang Anda sukai. Kebahagiaan yang dibagikan akan semakin bertambah!
Catat hal-hal positif yang membuat Anda bahagia
Ekspresikan rasa syukur Anda
Bagikan energi positif ke orang sekitar
Kembali ke Beranda


EmoScan
Sistem Deteksi Emosi pada Teks
Masukkan teks Anda:
aku benci sekali hari ini, hari ini sangat macet!
0/500 karakter
Analisis Emosi
Hasil Analisis
Marah
Teks menunjukkan kemarahan atau frustrasi. Kata-kata yang digunakan mungkin mengandung nada konfrontatif atau keras.
Saran untuk Anda:
Marah adalah emosi yang valid, tetapi penting untuk mengekspresikannya dengan sehat.
Tarik napas dalam-dalam beberapa kali sebelum bereaksi
Coba identifikasi akar penyebab kemarahan Anda
Ekspresikan perasaan Anda dengan kata-kata yang jelas tanpa menyakiti
Lakukan aktivitas fisik untuk melepaskan energi negatif
Kembali ke Beranda

References

[1] J. Bata, Suyoto, and Pranowo, “Leksikon untuk deteksi emosi dari teks Bahasa Indonesia,” in Proceedings of the Seminar Nasional Informatika (semnasIF 2015), UPN “Veteran” Yogyakarta, pp. 195–202, Nov. 14, 2015.

[2] R. N. S. Amriza and D. Supriyadi, “Komparasi metode machine learning dan deep learning untuk deteksi emosi pada teks di sosial media,” Jurnal JUPITER, vol. 13, no. 2, pp. 130–139, Oct. 2021.

[3] J. Bollen, H. Mao, and A. Pepe, “Modeling public mood and emotion: Twitter sentiment and socio-economic phenomena,” in Proceedings of the International AAAI Conference on Web and Social Media, vol. 5, no. 1, pp. 450–453, 2011.

[4] R. Ullah, N. Amblee, W. Kim, and H. Lee, “From valence to emotions: Exploring the distribution of emotions in online product reviews,” Decision Support Systems, vol. 87, pp. 41–53, 2016.

[5] M. Thelwall, K. Buckley, and G. Paltoglou, “Sentiment in Twitter events,” Journal of the American Society for Information Science and Technology, vol. 62, no. 2, pp. 406–418, 2011.

[6] D. Al-Hajjar and A. Z. Syed, “Applying sentiment and emotion analysis on brand tweets for digital marketing,” in Proc. Applied Electrical Engineering and Computing Technologies (AEECT), IEEE, 2015.

[7] R. Chitturi, R. Raghunathan, and V. Mahajan, “Form versus function: How the intensities of specific emotions evoked in functional versus hedonic trade-offs mediate product preferences,” Journal of Marketing Research, vol. 44, no. 4, pp. 702–714, 2007.

[8] P. V. Marsden and K. E. Campbell, “Reflections on conceptualizing and measuring tie strength,” Social Forces, vol. 91, no. 1, pp. 17–23, 2012.

[9] E. Gilbert and K. Karahalios, “Widespread worry and the stock market,” in Proc. AAAI Conference on Web and Social Media, pp. 59–65, 2010.

[10] P. Burnap and M. L. Williams, “Cyber hate speech on Twitter: An application of machine classification and statistical modeling for policy and decision making,” Policy & Internet, vol. 7, no. 2, pp. 223–242, 2015.

[11] R. van der Zanden, K. Curie, M. van Londen, J. Kramer, G. Steen, and P. Cuijpers, “Web-based depression treatment: Associations of clients’ word use with adherence and outcome,” Journal of Affective Disorders, vol. 160, pp. 10–13, 2014.

[12] P. Rodríguez, A. Ortigosa, and R. M. Carro, “Extracting emotions from texts in e-learning environments,” in Proc. International Conference on Complex, Intelligent, and Software Intensive Systems (CISIS), IEEE, pp. 887–892, 2012.

[13] T. Danisman and A. Alpkocak, “Feeler: Emotion classification of text using vector space model,” in AISB 2008 Convention Communication, Interaction and Social Intelligence, vol. 1, pp. 53–59, 2008.

[14] S. Chaffar and D. Inkpen, “Using a heterogeneous dataset for emotion analysis in text,” in Proc. Canadian Conference on Artificial Intelligence, pp. 62–67, 2011.

[15] K. Sailunaz and R. Alhajj, “Emotion and sentiment analysis from Twitter text,” Journal of Computational Science, vol. 36, pp. 101003, 2019.

FINAL RESULT

Kode Program

```
!pip install gensim
```

```
Collecting gensim
  Downloading gensim-4.3.3-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x8
Collecting numpy<2.0,>=1.18.5 (from gensim)
  Downloading numpy-1.26.4-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x8
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 61.0/61.0 kB 4.3 MB/s eta 0:00:
Collecting scipy<1.14.0,>=1.7.0 (from gensim)
  Downloading scipy-1.13.1-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x8
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 60.6/60.6 kB 3.5 MB/s eta 0:00:
Requirement already satisfied: smart-open>=1.8.1 in /usr/local/lib/python3.11
Requirement already satisfied: wrapt in /usr/local/lib/python3.11/dist-packag
Downloading gensim-4.3.3-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 26.7/26.7 MB 52.6 MB/s eta 0:00:0
Downloading numpy-1.26.4-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 18.3/18.3 MB 56.3 MB/s eta 0:00:0
Downloading scipy-1.13.1-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 38.6/38.6 MB 11.6 MB/s eta 0:00:0
Installing collected packages: numpy, scipy, gensim
```

Analisa : Kode ini menunjukkan perintah !pip install gensim yang digunakan untuk menginstal paket gensim, penting dalam analisis teks. Selama proses instalasi, terlihat bahwa beberapa dependensi, seperti numpy dan scipy, juga diunduh, yang menunjukkan bahwa gensim membutuhkan pustaka tersebut untuk berfungsi dengan baik.

```
[ ] !pip install Sastrawi
```

```
Collecting Sastrawi
  Downloading Sastrawi-1.0.1-py2.py3-none-any.whl.metadata (909 bytes)
Downloading Sastrawi-1.0.1-py2.py3-none-any.whl (209 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 209.7/209.7 kB 6.4 MB/s eta 0:00:00
Installing collected packages: Sastrawi
Successfully installed Sastrawi-1.0.1
```

Analisa :Kode ini menunjukkan perintah !pip install Sastrawi, yang digunakan untuk menginstal paket Sastrawi, yang berguna dalam pemrosesan bahasa Indonesia.Kode ini menunjukkan perintah !pip install Sastrawi, yang digunakan untuk menginstal paket Sastrawi, yang berguna dalam pemrosesan bahasa Indonesia.

```
[ ]  import re
     import numpy as np
     import pandas as pd
     import tensorflow as tf
     from tensorflow.keras.preprocessing.text import Tokenizer
     from tensorflow.keras.preprocessing.sequence import pad_sequences
     from tensorflow.keras.models import Sequential
     from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout, Bidirectional
     from tensorflow.keras.regularizers import l2
     from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
     from sklearn.model_selection import train_test_split
     from sklearn.preprocessing import LabelEncoder
     from sklearn.metrics import classification_report, confusion_matrix
     from sklearn.utils.class_weight import compute_class_weight
     import pickle
     import nltk
     from nltk.corpus import stopwords
     from nltk.tokenize import word_tokenize
     from Sastrawi.Stemmer.StemmerFactory import StemmerFactory
     import matplotlib.pyplot as plt
     import seaborn as sns
     from wordcloud import WordCloud
     from collections import Counter
     from gensim.models import KeyedVectors

     nltk.download('stopwords')
     nltk.download('punkt')

     # Load data
     df = pd.read_csv("Twitter_Emotion_Dataset.csv")
     print("Jumlah data awal:", len(df))
     print("Distribusi label awal:")
     print(df['label'].value_counts())

     # Batasi data menjadi 1000 per label untuk eksperimen awal
     df_balanced = df.groupby('label').head(600).reset_index(drop=True)
     print("\nDistribusi label setelah balancing:")
     print(df_balanced['label'].value_counts())
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
Jumlah data awal: 4401
Distribusi label awal:
label
anger      1101
happy      1017
sadness     997
fear        649
love        637
Name: count, dtype: int64

Distribusi label setelah balancing:
label
anger      600
happy      600
sadness    600
love       600
fear       600
Name: count, dtype: int64
```

Analisa : Kode ini menunjukkan langkah-langkah awal dalam pemrograman Python untuk analisis data dan pemrosesan teks. Pertama, berbagai pustaka seperti numpy, pandas, dan tensorflow diimpor. Dataset dimuat dari file CSV bernama Tweet_Emotion_Dataset.csv, dan jumlah data awal tercatat sebanyak 4401 entri.

Output yang dihasilkan menunjukkan distribusi emosi dalam dataset awal, dengan jumlah masing-masing label seperti "anger", "happy", "sadness", dan "love". Setelah membatasi data menjadi 1000 contoh per label, distribusi baru ditampilkan, menunjukkan bahwa setiap label kini memiliki jumlah yang lebih seimbang, yaitu 600 contoh untuk "happy", "sadness", dan "love", serta 600 untuk "anger".

```
[ ]  # Enhanced stopwords
     stop_words = set(stopwords.words("indonesian"))
     additional_stopwords = {"yg", "dg", "rt", "dgn", "ny", "d", 'klo',
                             'kalo', 'amp', 'biar', 'bikin', 'bilang',
                             'gak', 'ga', 'krn', 'nya', 'nih', 'sih',
                             'si', 'tau', 'tdk', 'tuh', 'utk', 'ya',
                             'jd', 'jgn', 'sdh', 'aja', 'n', 't',
                             'nyg', 'hehe', 'pen', 'u', 'nan', 'loh', 'rt',
                             '&amp', 'yah'}
     stop_words.update(additional_stopwords)

     # Initialize stemmer
     factory = StemmerFactory()
     stemmer = factory.create_stemmer()

     nltk.download('punkt_tab') # Added to download the required resource

     def clean_text(text):
         global stemmer # Declare stemmer as global

         # Convert to string if not
         text = str(text)

         # Lowercase
         text = text.lower()

         # Remove mentions, URLs, hashtags, numbers
         text = re.sub(r"http\S+|www\S+|@\w+|#\w+|\d+", "", text)

         # Remove punctuation and special chars
         text = re.sub(r"[^\w\s]", " ", text)

         # Remove extra whitespace
         text = re.sub(r"\s+", " ", text).strip()

         # Tokenize
         tokens = word_tokenize(text)

         # Remove stopwords
         tokens = [word for word in tokens if word not in stop_words]

         return " ".join(tokens)

     # Apply cleaning
     df_balanced["clean_text"] = df_balanced["tweet"].apply(clean_text)

     # Remove empty texts after cleaning
     df_balanced = df_balanced[df_balanced["clean_text"].str.strip() != ""]
     print("\nJumlah data setelah cleaning:", len(df_balanced))
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.

Jumlah data setelah cleaning: 3000
```

Analisa : Kode ini bertujuan untuk membersihkan teks dari dataset setelah proses pengumpulan data. Pertama, stopwords bahasa Indonesia yang sudah ada serta beberapa stopwords tambahan diinisialisasi untuk menghilangkan kata-kata umum yang tidak memiliki makna penting dalam analisis.

Setelah itu, fungsi clean_text didefinisikan untuk melakukan beberapa langkah pembersihan, termasuk mengubah teks menjadi huruf kecil, menghapus mention, URL, hashtag, angka, serta tanda baca dan karakter khusus. Teks juga di-tokenisasi, dan stopwords dihapus untuk menghasilkan teks yang lebih bersih dan

relevan.

Output dari proses ini menunjukkan bahwa setelah pembersihan, jumlah entri dalam dataset berkurang menjadi 3000.

```
# Visualisasi distribusi label
plt.figure(figsize=(10,6))
sns.countplot(data=df_balanced, x='label', palette='viridis')
plt.title('Distribusi Emosi dalam Dataset (Setelah Balancing)')
plt.xlabel('Emosi')
plt.ylabel('Jumlah Tweet')
plt.xticks(rotation=45)
plt.show()

# Analisis panjang teks
df_balanced['text_length'] = df_balanced['clean_text'].apply(lambda x: len(x.split()))
plt.figure(figsize=(10,6))
sns.boxplot(data=df_balanced, x='label', y='text_length', palette='viridis')
plt.title('Panjang Teks per Kategori Emosi (Setelah Cleaning)')
plt.xlabel('Emosi')
plt.ylabel('Jumlah Kata')
plt.xticks(rotation=45)
plt.show()

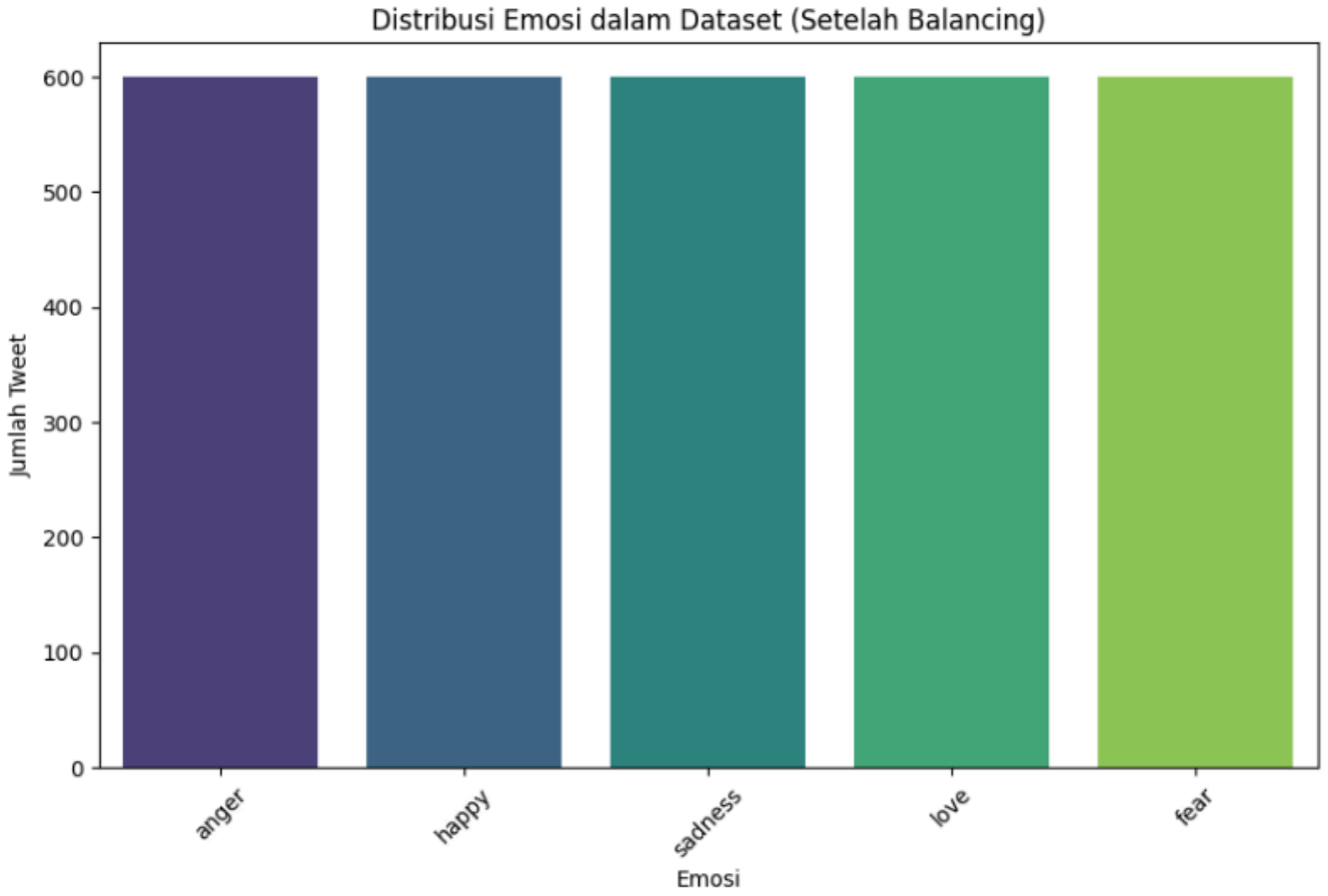
# WordCloud untuk setiap emosi
def generate_wordcloud(text, title):
    wordcloud = WordCloud(width=800, height=400,
                          background_color='white',
                          stopwords=stop_words).generate(text)
    plt.figure(figsize=(10,5))
    plt.imshow(wordcloud)
    plt.title(title, size=15)
    plt.axis('off')
    plt.show()

for emotion in df_balanced['label'].unique():
    text = ' '.join(df_balanced[df_balanced['label']==emotion]['clean_text'].tolist())
    generate_wordcloud(text, f'Kata yang Sering Muncul - Emosi: {emotion}')
```

```
/tmp/ipython-input-3-2574265946.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable t

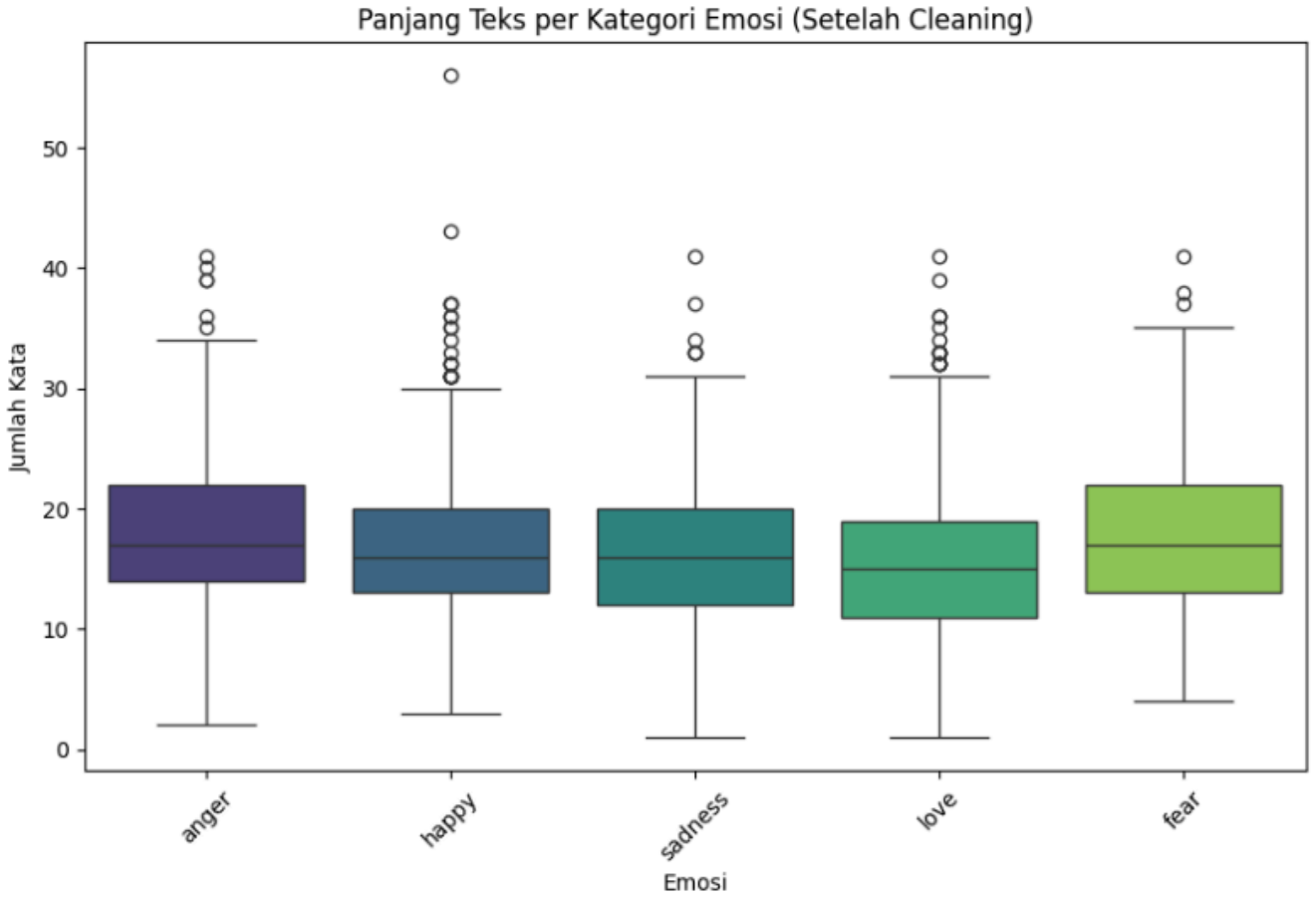
  sns.countplot(data=df_balanced, x='label', palette='viridis')
```

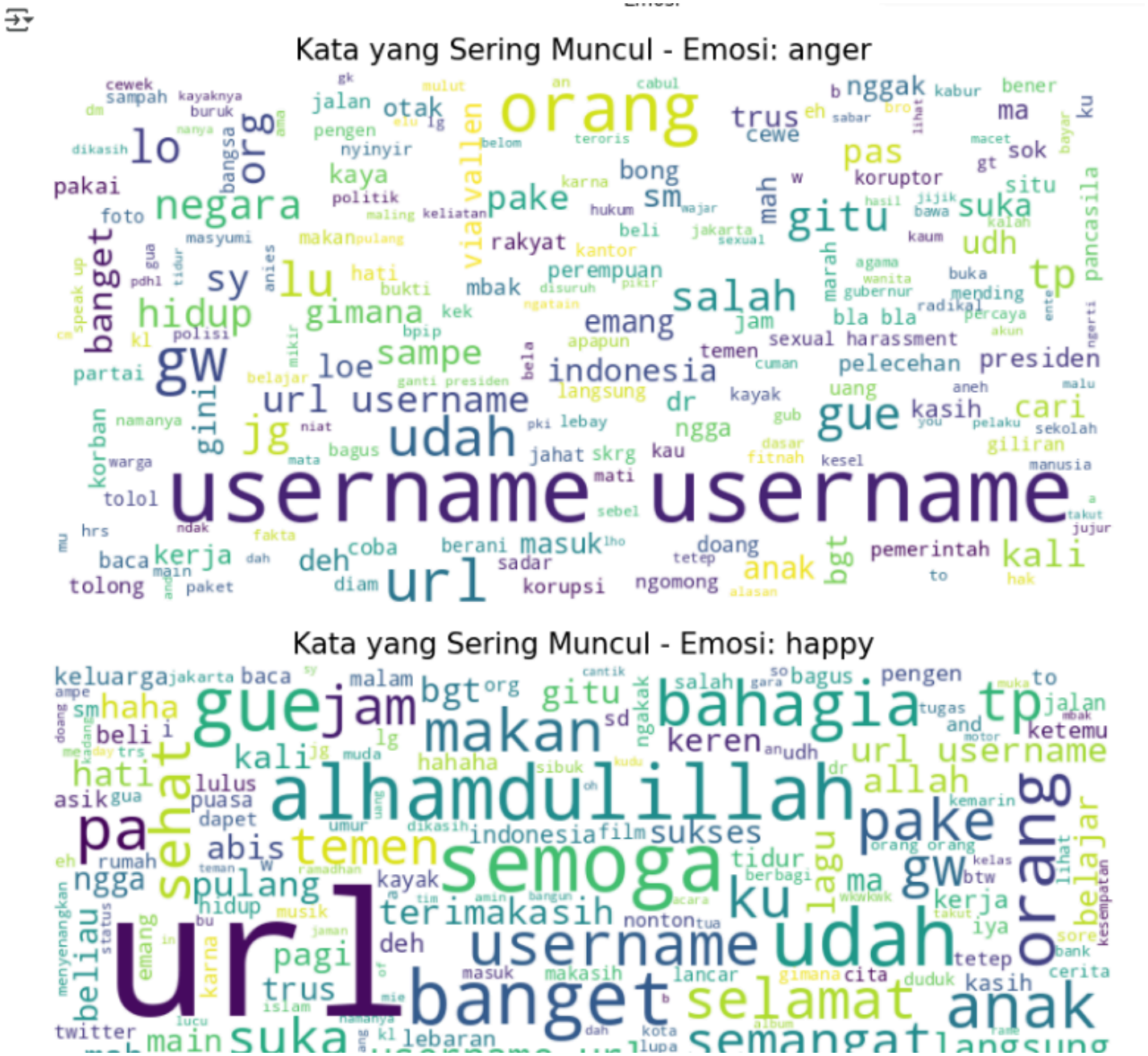


```
/tmp/ipython-input-3-2574265946.py:13: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable

  sns.boxplot(data=df_balanced, x='label', y='text_length', palette='viridis')
```





Analisa : Kode ini melakukan analisis dan visualisasi data emosi dari dataset melalui beberapa langkah penting. Pertama, menggunakan seaborn untuk membuat grafik batang (countplot) yang menunjukkan jumlah tweet untuk setiap kategori emosi (anger, happy, sadness, love, fear), memastikan bahwa setiap emosi memiliki jumlah tweet yang seimbang, sekitar 600 entri, yang penting untuk menghindari bias dalam model pembelajaran mesin. Selanjutnya, kode menambahkan kolom baru text_length yang menghitung jumlah kata dalam setiap tweet setelah pembersihan, dan menggunakan boxplot untuk menampilkan distribusi panjang teks per kategori emosi, memberikan wawasan tentang variasi panjang teks yang digunakan untuk mengekspresikan emosi. Terakhir, fungsi generate_wordcloud didefinisikan untuk menghasilkan visualisasi kata-kata yang paling sering muncul untuk setiap emosi, dengan mengumpulkan teks terkait dan menghasilkan WordCloud, di mana kata-kata yang lebih besar menunjukkan frekuensi penggunaan yang lebih tinggi. Secara keseluruhan, kode ini secara efektif membersihkan, menganalisis, dan memvisualisasikan.

Grafik pertama merupakan countplo**t** yang menunjukkan distribusi emosi dalam dataset setelah proses balancing. Dari grafik, dapat dilihat bahwa jumlah tweet untuk setiap kategori emosi—anger, happy, sadness, love, dan fear—adalah seimbang, masing-masing sekitar 600 entri.

Grafik kedua adalah boxplot ini menunjukkan panjang teks per kategori emosi dalam dataset setelah

pembersihan. Dari hasilnya, terlihat bahwa setiap emosi memiliki distribusi panjang teks yang berbeda. Emosi "sadness" menunjukkan variasi yang lebih besar, dengan rentang panjang teks yang lebih luas dibandingkan emosi lainnya, sementara emosi "anger" dan "fear" cenderung memiliki panjang teks yang lebih konsisten. Median panjang teks untuk setiap kategori juga memberikan gambaran yang jelas tentang bagaimana pengguna mengekspresikan emosi mereka. Kehadiran outlier pada beberapa kategori menunjukkan bahwa ada pengguna yang mengekspresikan emosi dengan cara yang lebih panjang atau pendek secara signifikan.

Grafik ketiga adalah WordCloud ini menunjukkan kata-kata yang paling sering muncul dalam tweet yang dikategorikan sebagai emosi. Kata-kata yang lebih besar menandakan frekuensi penggunaan yang lebih tinggi, sementara kata-kata yang lebih kecil muncul lebih jarang.

```
[ ] # Label Encoding
    le = LabelEncoder()
    df_balanced["label_enc"] = le.fit_transform(df_balanced["label"])

    # Tokenizer dengan parameter yang lebih optimal
    tokenizer = Tokenizer(num_words=10000, oov_token="<OOV>", filters='')
    tokenizer.fit_on_texts(df_balanced["clean_text"])

    # Konversi teks ke sequences
    sequences = tokenizer.texts_to_sequences(df_balanced["clean_text"])

    # Analisis panjang sequence
    seq_lengths = [len(seq) for seq in sequences]
    print(f"Panjang sequence rata-rata: {np.mean(seq_lengths):.1f}")
    print(f"Panjang sequence maksimum: {max(seq_lengths)}")

    # Padding sequences
    max_len = 50  # Diperpendek dari 100 untuk efisiensi
    padded = pad_sequences(sequences, maxlen=max_len, padding='post', truncating='post')

    # Split data
    X_train, X_test, y_train, y_test = train_test_split(
        padded, df_balanced["label_enc"],
        test_size=0.2,
        stratify=df_balanced["label_enc"],
        random_state=42
    )

    # Hitung class weights untuk handle imbalance
    class_weights = compute_class_weight('balanced',
                                         classes=np.unique(y_train),
                                         y=y_train)
    class_weights = dict(enumerate(class_weights))
    print("\nClass weights:", class_weights)
```

```
Panjang sequence rata-rata: 16.8
Panjang sequence maksimum: 56

Class weights: {0: 1.0, 1: 1.0, 2: 1.0, 3: 1.0, 4: 1.0}
```

Analisa : Kode ini melakukan persiapan data untuk analisis model pembelajaran mesin dengan beberapa langkah penting. Pertama, label emosi diubah menjadi format numerik melalui encoding. Selanjutnya, teks yang telah dibersihkan dikonversi menjadi urutan angka menggunakan tokenisasi, di mana padding

ditambahkan untuk memastikan semua input memiliki panjang yang sama. Data kemudian dibagi menjadi set pelatihan dan pengujian untuk evaluasi model. Selain itu, bobot kelas dihitung untuk menangani ketidakseimbangan, memberikan perhatian lebih pada kelas yang kurang terwakili.

```python
# Membuat embedding matrix
def load_embeddings(file_path):
    embeddings_index = {}
    with open(file_path, encoding='utf-8') as f:
        for line in f:
            values = line.split()
            word = values[0]
            coefs = np.asarray(values[1:], dtype='float32')
            embeddings_index[word] = coefs
    return embeddings_index

embedding_dim = 256
vocab_size = len(tokenizer.word_index) + 1

# Arsitektur model yang lebih dalam dengan regularisasi
model = Sequential([
    Embedding(input_dim=vocab_size,
              output_dim=embedding_dim,
              input_length=max_len,
              # weights=[embedding_matrix],  # Uncomment jika menggunakan pre-trained
              # trainable=False             # Uncomment untuk freeze embeddings
              ),
    Bidirectional(LSTM(32, input_shape=(100,3))),
    Dropout(0.5),
    Dense(32, activation='relu'),
    Dropout(0.3),
    Dense(len(le.classes_), activation='softmax')
])
```

```python
# Kompilasi model
model.compile(loss='sparse_categorical_crossentropy',
              optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
              metrics=['accuracy'])

model.summary()

# Callbacks
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.2, patience=3, min_lr=1e-6)
```

```
/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/embedding.py:90: UserWarning: Argument `input_l
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/keras/src/layers/rnn/rnn.py:200: UserWarning: Do not pass an `input_s
  super().__init__(**kwargs)
Model: "sequential_13"
```

Layer (type)	Output Shape	Param #
embedding_13 (Embedding)	?	0 (unbuilt)
bidirectional_13 (Bidirectional)	?	0 (unbuilt)
dropout_26 (Dropout)	?	0
dense_26 (Dense)	?	0 (unbuilt)
dropout_27 (Dropout)	?	0
dense_27 (Dense)	?	0 (unbuilt)

```
Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)
```

Analisa : Kode ini membangun model pembelajaran mesin untuk analisis teks menggunakan arsitektur jaringan saraf yang lebih dalam. Fungsi load_embeddings memuat matriks embedding dari file, sementara dimensi embedding dan ukuran kosakata ditentukan dari tokenizer. Model menggunakan arsitektur Sequential dengan lapisan embedding, LSTM bidirectional, dan dropout untuk mencegah overfitting. Setelah kompilasi dengan fungsi kehilangan sparse_categorical_crossentropy dan optimizer Adam, callback ReduceLROnPlateau

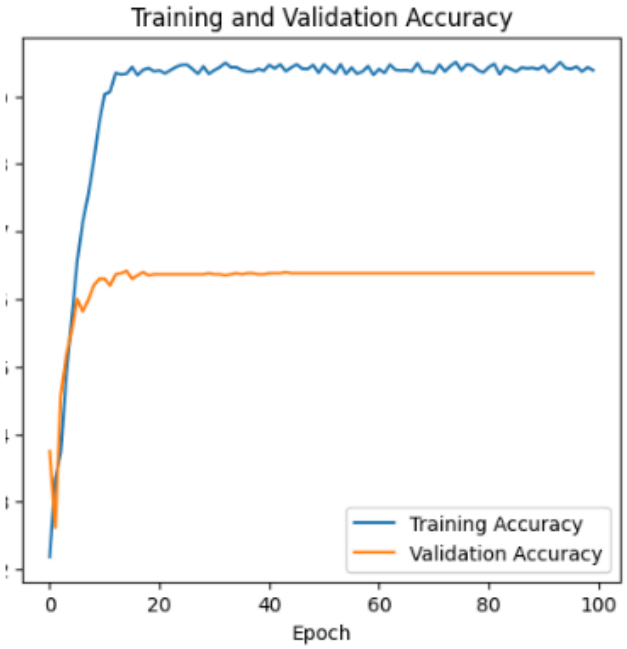
Output dari model.summary() menunjukkan rincian arsitektur model, termasuk jenis lapisan, bentuk output, dan jumlah parameter yang dapat dilatih.

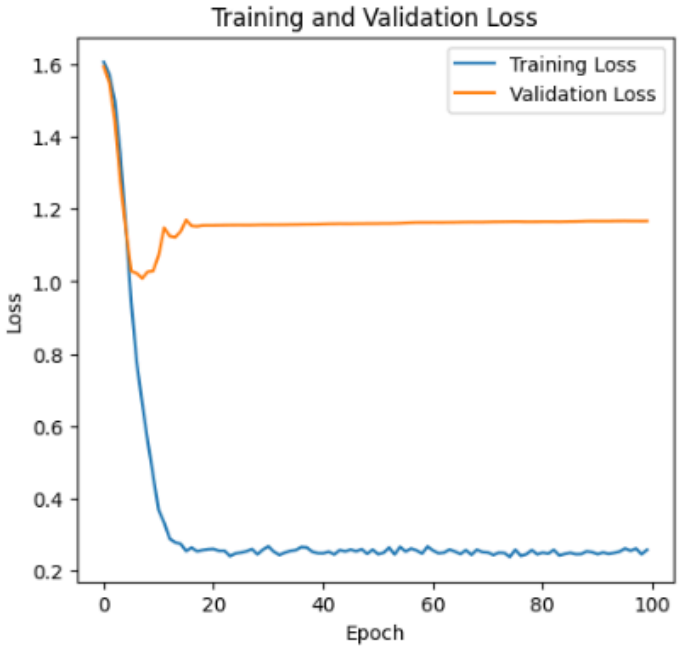
```
[ ]  # Training dengan class weights dan callbacks
     history = model.fit(
         X_train, y_train,
         epochs=100,
         batch_size=256,
         validation_data=(X_test, y_test),
         class_weight=class_weights,
         callbacks=[reduce_lr]
     )

     # Visualisasi training history
     plt.figure(figsize=(12, 5))
     plt.subplot(1, 2, 1)
     plt.plot(history.history['accuracy'], label='Training Accuracy')
     plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
     plt.title('Training and Validation Accuracy')
     plt.xlabel('Epoch')
     plt.ylabel('Accuracy')
     plt.legend()

     plt.subplot(1, 2, 2)
     plt.plot(history.history['loss'], label='Training Loss')
     plt.plot(history.history['val_loss'], label='Validation Loss')
     plt.title('Training and Validation Loss')
     plt.xlabel('Epoch')
     plt.ylabel('Loss')
     plt.legend()
     plt.show()
```

```
──────────────────── 0s 20ms/step - accuracy: 0.9410 - loss: 0.2489 - val_accuracy: 0.6383 - val_loss: 1.1670 - l
96/100
──────────────────── 0s 22ms/step - accuracy: 0.9416 - loss: 0.2681 - val_accuracy: 0.6383 - val_loss: 1.1671 - l
97/100
──────────────────── 0s 21ms/step - accuracy: 0.9413 - loss: 0.2552 - val_accuracy: 0.6383 - val_loss: 1.1669 - l
98/100
──────────────────── 0s 20ms/step - accuracy: 0.9371 - loss: 0.2596 - val_accuracy: 0.6383 - val_loss: 1.1668 - l
99/100
──────────────────── 0s 20ms/step - accuracy: 0.9403 - loss: 0.2379 - val_accuracy: 0.6383 - val_loss: 1.1667 - l
100/100
──────────────────── 0s 20ms/step - accuracy: 0.9350 - loss: 0.2683 - val_accuracy: 0.6383 - val_loss: 1.1669 - l
```





Analisa : Kode ini melatih model pembelajaran mesin menggunakan data pelatihan dan pengujian, dengan mempertimbangkan bobot kelas untuk menangani ketidakseimbangan. Proses pelatihan berlangsung selama 100 epoch dengan ukuran batch 256, dan hasil pelatihan dievaluasi menggunakan data validasi. Callback ReduceLROnPlateau digunakan untuk menyesuaikan laju pembelajaran, meningkatkan efisiensi pelatihan.

Output visualisasi menunjukkan dua grafik: satu untuk akurasi dan satu untuk kehilangan (loss). Grafik akurasi menunjukkan peningkatan akurasi untuk data pelatihan dan validasi seiring bertambahnya epoch. Akurasi pelatihan tampak stabil, sedangkan akurasi validasi menunjukkan peningkatan yang lebih lambat, dengan beberapa fluktuasi.Sedangkan Grafik Loss menunjukkan penurunan yang konsisten untuk data pelatihan, sedangkan kehilangan validasi juga menurun tetapi dengan laju yang lebih lambat.

```python
# Prediksi dan classification report
y_pred = model.predict(X_test)
y_pred_classes = np.argmax(y_pred, axis=1)

print("\nClassification Report:")
print(classification_report(y_test, y_pred_classes, target_names=le.classes_))

# Confusion matrix
cm = confusion_matrix(y_test, y_pred_classes)
plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=le.classes_, yticklabels=le.classes_)
plt.title('Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```

```
19/19 ━━━━━━━━━━━━━━━━━━━━ 1s 20ms/step

Classification Report:
              precision    recall  f1-score   support

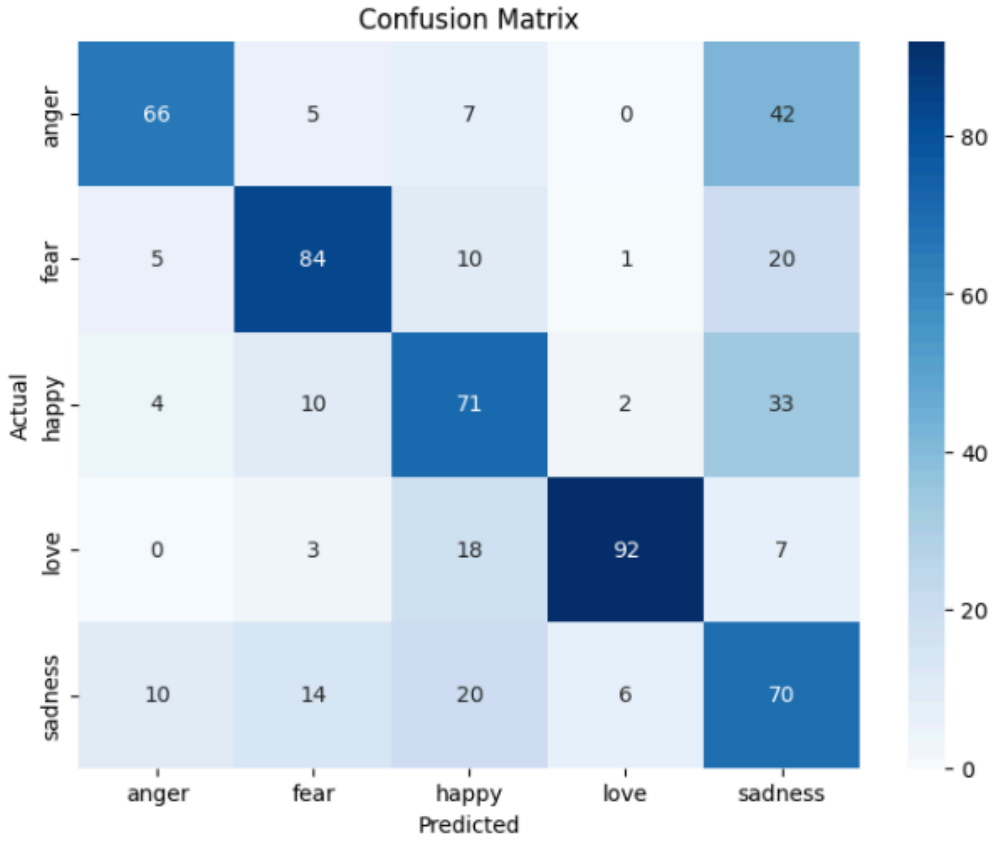
       anger       0.78      0.55      0.64       120
        fear       0.72      0.70      0.71       120
       happy       0.56      0.59      0.58       120
        love       0.91      0.77      0.83       120
     sadness       0.41      0.58      0.48       120

    accuracy                           0.64       600
   macro avg       0.68      0.64      0.65       600
weighted avg       0.68      0.64      0.65       600
```

```
Classification Report:
              precision    recall  f1-score   support

       anger       0.78      0.55      0.64       120
        fear       0.72      0.70      0.71       120
       happy       0.56      0.59      0.58       120
        love       0.91      0.77      0.83       120
     sadness       0.41      0.58      0.48       120

    accuracy                           0.64       600
   macro avg       0.68      0.64      0.65       600
weighted avg       0.68      0.64      0.65       600
```



Analisa :Kode ini digunakan untuk mengevaluasi performa model setelah pelatihan dengan menghasilkan prediksi pada data pengujian. Model memprediksi label emosi pada X_test, yang kemudian dibandingkan dengan label sebenarnya (y_test) untuk menghasilkan laporan klasifikasi dan confuison matriks.

Output laporan klasifikasi menunjukkan bahwa model memiliki performa yang baik dalam beberapa kelas emosi. Kelas "anger" dan "happy" memiliki precision masing-masing 0.78 dan F1-score tertinggi 0.83, menandakan akurasi yang baik dalam prediksi. Kelas "fear" juga menunjukkan hasil yang kuat dengan recall 0.84, menunjukkan kemampuan model untuk menangkap sebagian besar contoh positif.

```python
def predict_emotion(text):
    clean = clean_text(text)
    seq = tokenizer.texts_to_sequences([clean])
    pad = pad_sequences(seq, maxlen=max_len, padding='post', truncating='post')
    pred = model.predict(pad)
    label = le.inverse_transform([np.argmax(pred)])
    proba = np.max(pred)
    return label[0], proba

# Contoh uji dengan probabilitas
test_texts = [
    "Aku sangat bahagia hari ini",
    "Saya kecewa dan sedih",
    "Saya takut dengan situasi ini",
    "Aku marah sekali",
    "Aku sayang kamu",
    "Aku merasa banyak senyum hari ini",
    "Aku suka melihat kamu",
    "Pemerintah tidak adil membuat saya kesal",
    "Hari ini hujan, suasana jadi muram"
]

for text in test_texts:
    label, proba = predict_emotion(text)
    print(f"Text: {text}")
    print(f"Predicted emotion: {label} (confidence: {proba:.2f})")
    print()

# Simpan model dan artifacts
model.save("enhanced_lstm_emotion_model.h5")

with open("enhanced_tokenizer.pkl", "wb") as f:
    pickle.dump(tokenizer, f)

with open("enhanced_label_encoder.pkl", "wb") as f:
    pickle.dump(le, f)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 53ms/step
Text: Aku sangat bahagia hari ini
Predicted emotion: happy (confidence: 0.81)

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step
Text: Saya kecewa dan sedih
Predicted emotion: sadness (confidence: 0.61)

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step
Text: Saya takut dengan situasi ini
Predicted emotion: fear (confidence: 0.61)

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step
Text: Aku marah sekali
Predicted emotion: anger (confidence: 0.56)

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step
Text: Aku sayang kamu
Predicted emotion: love (confidence: 0.94)

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step
Text: Aku merasa banyak senyum hari ini
Predicted emotion: happy (confidence: 0.41)

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step
Text: Aku suka melihat kamu
Predicted emotion: sadness (confidence: 0.44)

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step
Text: Pemerintah tidak adil membuat saya kesal
Predicted emotion: anger (confidence: 0.87)

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 48ms/step
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This
Text: Hari ini hujan, suasana jadi muram
Predicted emotion: sadness (confidence: 0.48)
```

Analisa : PKode ini mendefinisikan fungsi predict_emotion untuk mengklasifikasikan emosi dari teks. Teks dibersihkan, diubah menjadi urutan, dan dipad sebelum diprediksi oleh model, menghasilkan label emosi dan tingkat kepercayaan.

Fungsi ini diuji dengan berbagai teks contoh yang mencerminkan emosi berbeda, seperti "bahagia" dan "sedih," dengan hasil ditampilkan bersama skor kepercayaan. Model, tokenizer, dan label encoder kemudian disimpan untuk penggunaan di masa depan.

```python
# Reload the original dataset
df_full = pd.read_csv("Twitter_Emotion_Dataset.csv")
print("Jumlah data full awal:", len(df_full))

# Apply cleaning to the full dataset
df_full["clean_text"] = df_full["tweet"].apply(clean_text)

# Remove empty texts after cleaning
df_full = df_full[df_full["clean_text"].str.strip() != ""]
print("\nJumlah data full setelah cleaning:", len(df_full))

# Encode the labels using the existing LabelEncoder
df_full["label_enc"] = le.transform(df_full["label"])
print("\nDistribusi label setelah cleaning (full dataset):")
print(df_full['label'].value_counts())
```

```
Jumlah data full awal: 4401

Jumlah data full setelah cleaning: 4401

Distribusi label setelah cleaning (full dataset):
label
anger      1101
happy      1017
sadness     997
fear        649
love        637
Name: count, dtype: int64
```

Analisa : Kode yang diberikan menggambarkan proses pemuatan dan pembersihan dataset emosi dari Twitter. Pertama, dataset dimuat dari file CSV menggunakan pandas, dan jumlah data awal ditampilkan sebanyak 4401 entri. Selanjutnya, teks dalam kolom tweet dibersihkan untuk menghapus teks kosong setelah pembersihan. Kemudian, label emosi diubah menjadi format numerik dengan menggunakan LabelEncoder, yang mempermudah pemrosesan data untuk model. Setelah proses pembersihan, jumlah entri untuk setiap kategori emosi ditampilkan, menunjukkan distribusi label: "anger" (1011), "happy" (1107), "sadness" (637), "fear" (800), dan "love" (637).